

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Микропроцессорные техника в мехатронике и
роботехнике»
ТЕМА: Работа с трехцветным светодиодом (ШИМ)
Вариант 8

Студент гр. 2491

Преподаватель




Недовизий А.И.
Сулинский Е.В.

Копычев М.М.

Санкт-Петербург

Задание на лабораторную работу

Написать программу управления цветом и яркостью светодиода с помощью механического энкодера. В таблице заданий 0....255 означает, что при вращении энкодера по часовой стрелке яркость данного цвета увеличивается, а против часовой стрелки – уменьшается. На ЖКИ выводятся значения яркости (0...255) для каждого из трёх каналов RGB диода в виде: R: xxx G: xxx B: xxx.

Основные теоретические положения

В данной лабораторной работе мы рассмотрим режим генерации ШИМ сигнала с фиксированной частотой для 16-ти битного таймера.

Управление 16-битными таймерами/счётчиками. 16-битные таймеры Atmega имеют 3 независимых канала ШИМ – А, В и С. Для установки режима работы таймера используются регистры TCCRxA и TCCRxB (x – 1 или 3).

В МК сигнал ШИМ формируется при сравнении значения регистра TCNT с заранее заданным значением из регистра OCR.

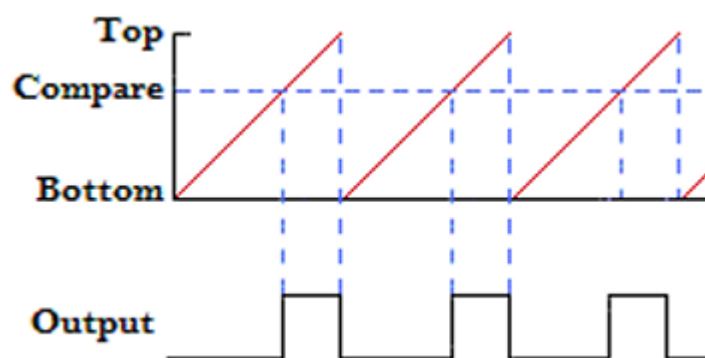


Рисунок 1 – ШИМ сигнал

Схема к лабораторной работе приведена на рисунке 2.

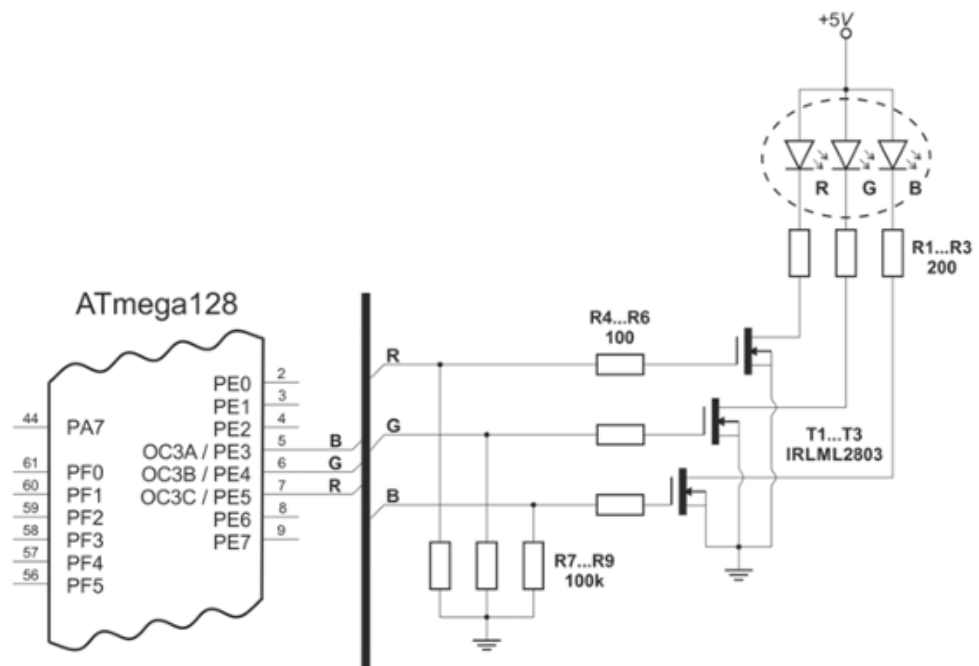


Рисунок 2 – Схема для лабораторной работы

Блок-схемы

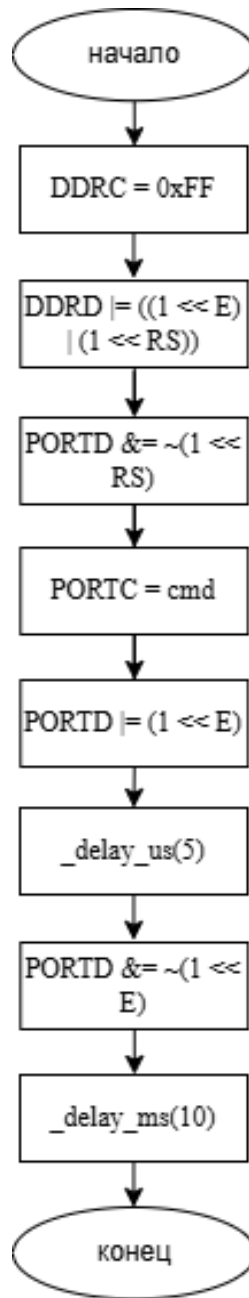


Рисунок 3 – Блок-схема функции lcdCmd()

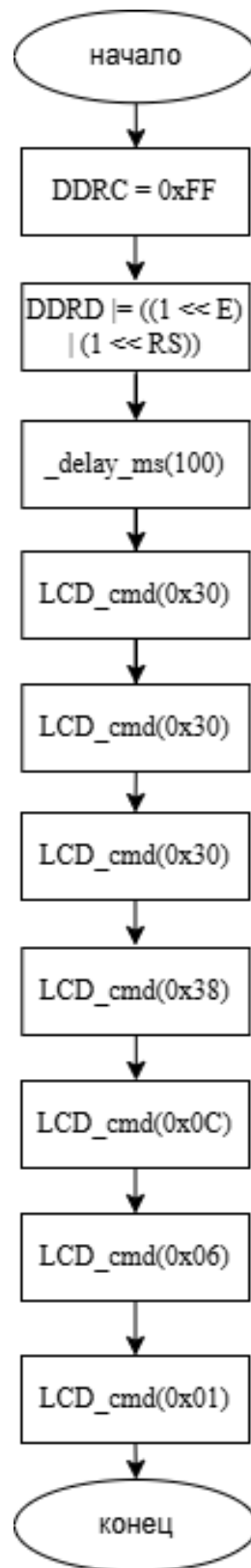


Рисунок 4 – Блок-схема функции lcdInit()

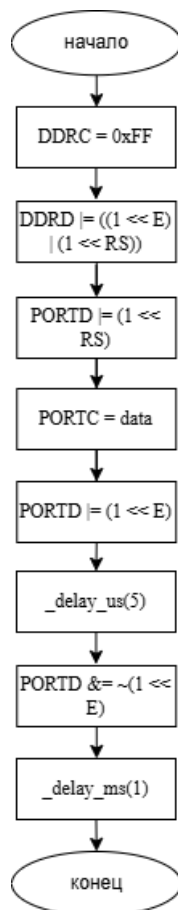


Рисунок 5 – Блок-схема функции lcdData

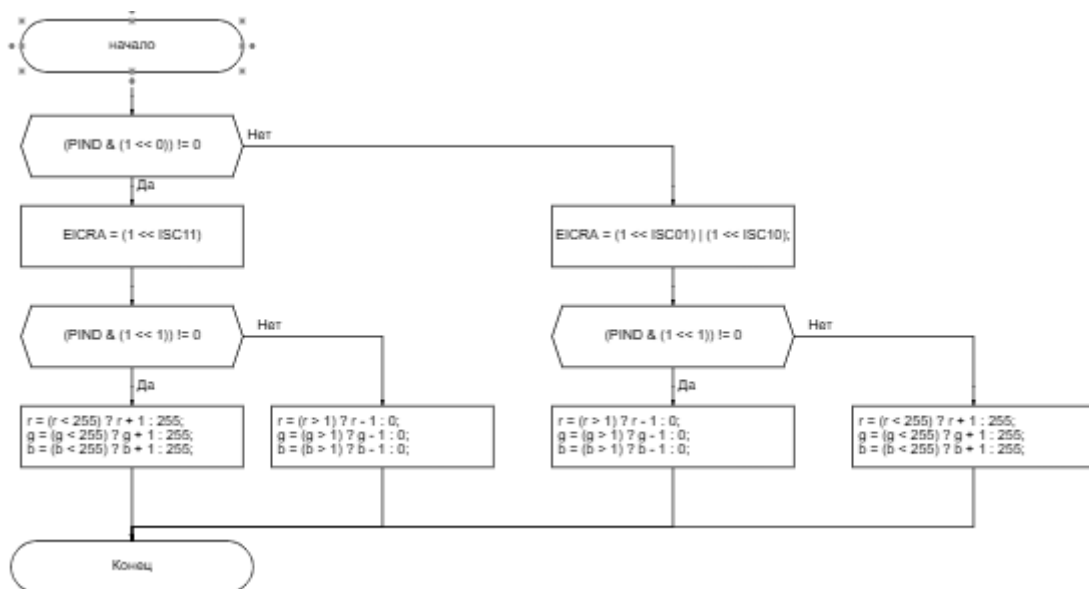


Рисунок 6 – Блок-схема функции ISR()

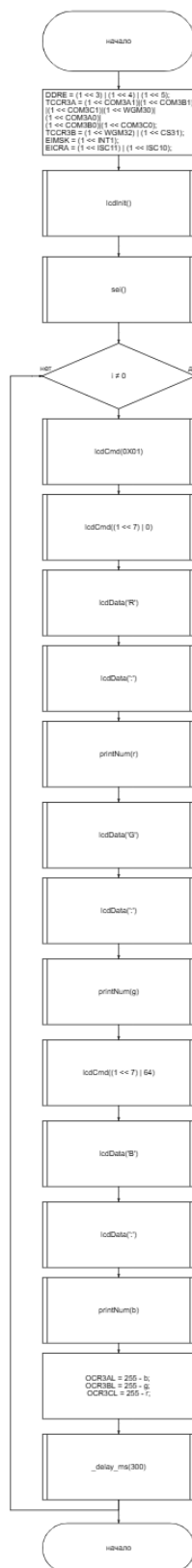


Рисунок 7 – Блок-схема функции main()

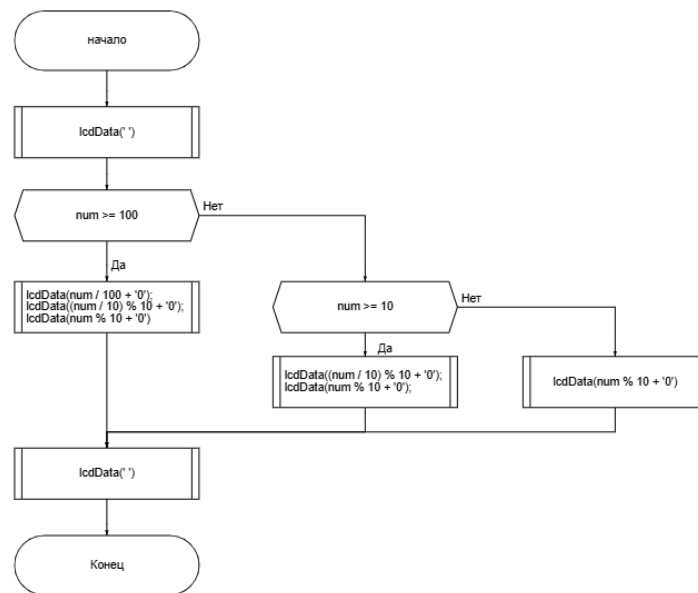


Рисунок 8 – Блок-схема функции printNum()

ВЫВОДЫ

В ходе выполнения лабораторной работы №2 была успешно разработана и реализована программа управления RGB-светодиодом с использованием механического энкодера для плавного изменения цвета и яркости. Программа комплексно объединила несколько ключевых аспектов программирования микроконтроллеров AVR: работу с внешними прерываниями, генерацию ШИМ-сигналов и управление жидкокристаллическим дисплеем.

Основным достижением стала корректная настройка таймера/счетчика №3 в режиме быстрой 8-битной ШИМ, что позволило независимо управлять интенсивностью свечения каждого из трех каналов RGB-светодиода. Через регистры OCR3A, OCR3B и OCR3C осуществляется точное регулирование коэффициента заполнения ШИМ-сигналов, что обеспечивает плавные переходы между цветами. Одновременно с этим была реализована обработка прерывания INT0 от механического энкодера, где в обработчике прерывания не только определяется направление вращения, но и динамически перенастраивается чувствительность прерывания для надежного детектирования обоих фронтов сигнала. Это позволило создать интуитивно понятный интерфейс управления.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <math.h>

#define E 6
#define RS 7

void lcdCmd(uint8_t cmd);
void lcdInit(void);
void lcdData(uint8_t data);
void printNum(uint16_t num);

uint8_t r = 0;
uint8_t g = 255;
uint8_t b = 255;

int main(void)
{
    DDRE = (1 << 3) | (1 << 4) | (1 << 5);
    /* Инициализация таймера №3. 16 битная быстрая
    ШИМ, преддедитель на 8 */
    TCCR3A = (1 << COM3A1) | (1 << COM3B1) | (1 << COM3C1) | (1 <<
WGM30) | (1 << COM3A0) | (1 << COM3B0) | (1 << COM3C0);
    TCCR3B = (1 << WGM32) | (1 << CS31);
    EIMSK = (1 << INT1);
    EICRA = (1 << ISC11) | (1 << ISC10);
    lcdInit();
    sei();
    while (1)
```

```

    {
        lcdCmd(0X01);
        lcdCmd((1 << 7) | 0);
        lcdData('R');
        lcdData(':');
        printNum(r);

        lcdData('G');
        lcdData(':');
        printNum(g);

        lcdCmd((1 << 7) | 64);
        lcdData('B');
        lcdData(':');
        printNum(b);

        OCR3AL = 255 - b;
        OCR3BL = 255 - g;
        OCR3CL = 255 - r;
        _delay_ms(300);
    }
}

void printNum(uint16_t num)
{
    lcdData(' ');
    if (num >= 100)
    {
        lcdData(num / 100 + '0');
        lcdData((num / 10) % 10 + '0');
        lcdData(num % 10 + '0');
    }
}

```

```

else if (num >= 10)
{
    lcdData((num / 10) % 10 + '0');
    lcdData(num % 10 + '0');
}
else
    lcdData(num % 10 + '0');

    lcdData(' ');
}

```

```

void lcdData(uint8_t data)
{ // посыл данных на экран
    DDRC = 0xFF;
    DDRD |= ((1 << E) | (1 << RS));
    PORTD |= (1 << RS);
    PORTC = data;
    PORTD |= (1 << E);
    _delay_us(5);
    PORTD &= ~(1 << E);
    _delay_ms(4);
}

```

```

void lcdInit(void)
{ // инициализация (ВКЛ) экрана
    DDRC = 0xFF; // все разряды PORTC на выход
    DDRD |= ((1 << E) | (1 << RS)); // разряды PORTD на выход
    _delay_ms(100); // задержка для установления питания
    lcdCmd(0x30); // \ вывод
    lcdCmd(0x30); // | трех
    lcdCmd(0x30); // / команд 0x30
    lcdCmd(0x38); // 8 разр.шина, 2 строки, 5 × 7 точек
}

```

```

    lcdCmd(0x0C); // включить ЖКИ
    lcdCmd(0x06); // инкремент курсора, без сдвига экрана
    lcdCmd(0x01); // очистить экран, курсор в начало
}

void lcdCmd(uint8_t cmd)
{ // посыл команды на экран
    DDRC = 0xFF; // все разряды PORTC на выход
    DDRD |= ((1 << E) | (1 << RS)); // разряды PORTD на выход
    PORTD &= ~(1 << RS); // выбор регистра команд RS=0
    PORTC = cmd; // записать команду в порт PORTC
    PORTD |= (1 << E); // \ сформировать на
    _delay_us(5); // | выводе E строб 1-0
    PORTD &= ~(1 << E); // / передачи команды
    _delay_ms(4); // задержка для завершения записи
}

ISR(INT1_vect)
{
    if ((PIND & (1 << 0)) != 0)
    {
        EICRA = (1 << ISC11);
        if ((PIND & (1 << 1)) != 0)
        {
            r = (r < 255) ? r + 1 : 255;
            g = (g < 255) ? g + 1 : 255;
            b = (b < 255) ? b + 1 : 255;
        }
        else
        {
            r = (r > 1) ? r - 1 : 0;
            g = (g > 1) ? g - 1 : 0;
            b = (b > 1) ? b - 1 : 0;
        }
    }
}

```

```

    }
}
else
{
    EICRA = (1 << ISC01) | (1 << ISC10);
    if ((PIND & (1 << 1)) != 0)
    {
        r = (r > 1) ? r - 1 : 0;
        g = (g > 1) ? g - 1 : 0;
        b = (b > 1) ? b - 1 : 0;
    }
    else
    {
        r = (r < 255) ? r + 1 : 255;
        g = (g < 255) ? g + 1 : 255;
        b = (b < 255) ? b + 1 : 255;
    }
}
}

```